

CSE 1201
Data Structure

Chapter 4

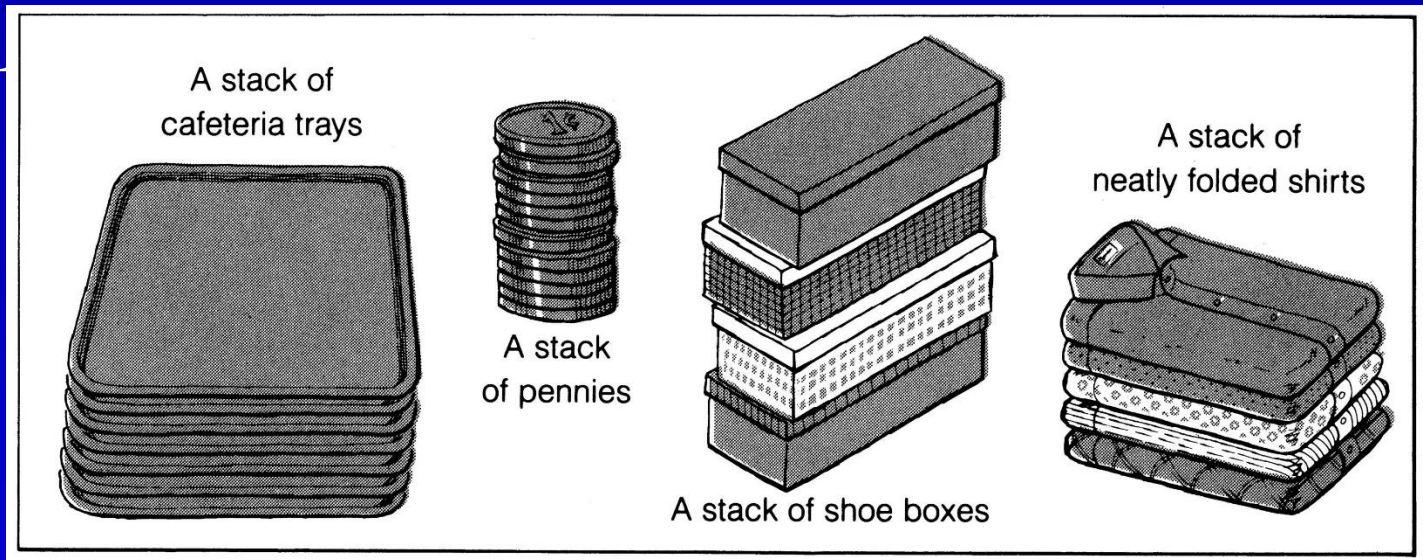
Stacks

Md. Nazrul Islam Mondal

What is a stack?

- It is an ordered group of homogeneous items or elements.
- Elements are added to and removed from the top of the stack (the most recently added items are at the top of the stack).
- The last element to be added is the first to be removed

(L



Stack Specification

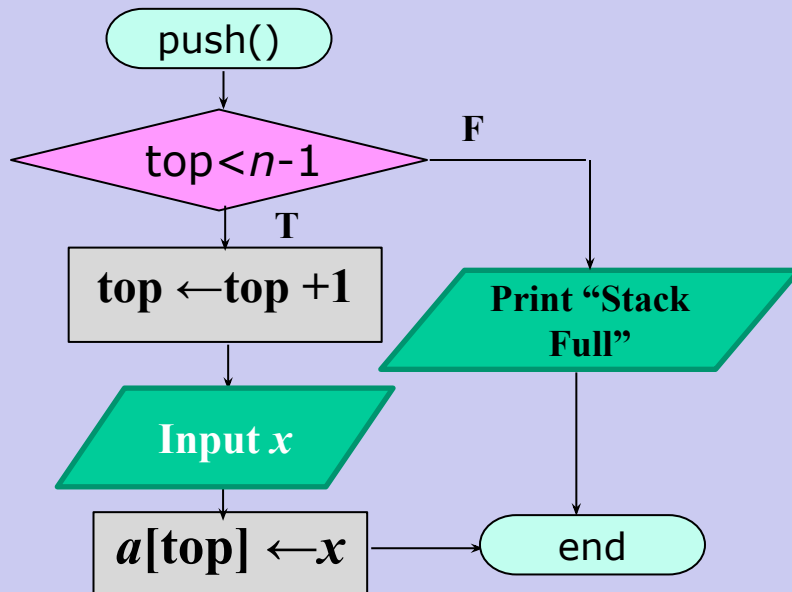
- Definitions: (provided by the user)
 - *MAX_ITEMS*: Max number of items that might be on the stack
 - *ItemType*: Data type of the items on the stack
- Operations
 - Push (ItemType newItem)
 - Pop ()

Push (ItemType newItem)

- *Function*: Adds newItem to the top of the stack.
- *Preconditions*: Stack has been initialized and is not full.
- *Postconditions*: newItem is at the top of the stack.

Stack: **push()** function

Topic 1: Write an Algorithm to push a new element in a stack



Initially top = -1

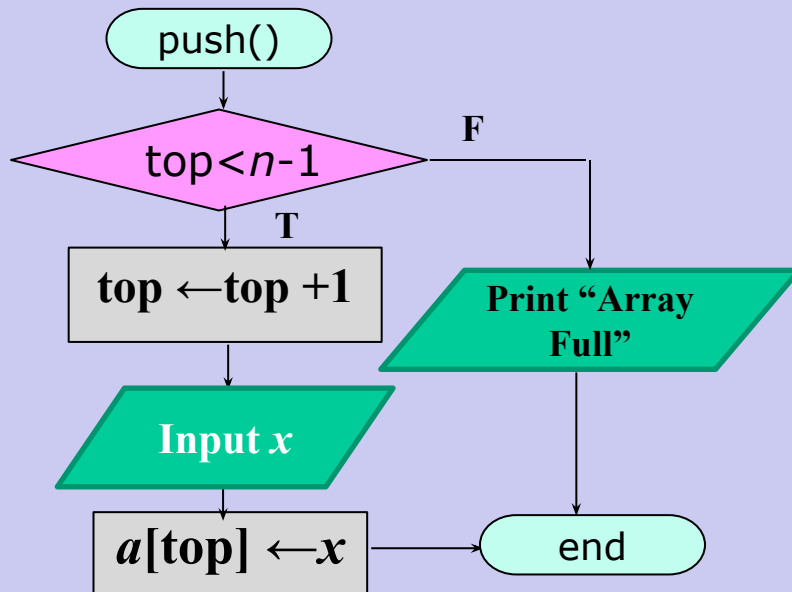
n : size of $a[]$
 x : input variable
 top : index of last input, declare it global.
Array Elements: $a[0] \dots a[n-1]$

$a[]$	
$n-1$	
...	
...	
2	
1	
0	

←top

Stack: **push()** function

Topic 1: Write an Algorithm to push a new element in a stack



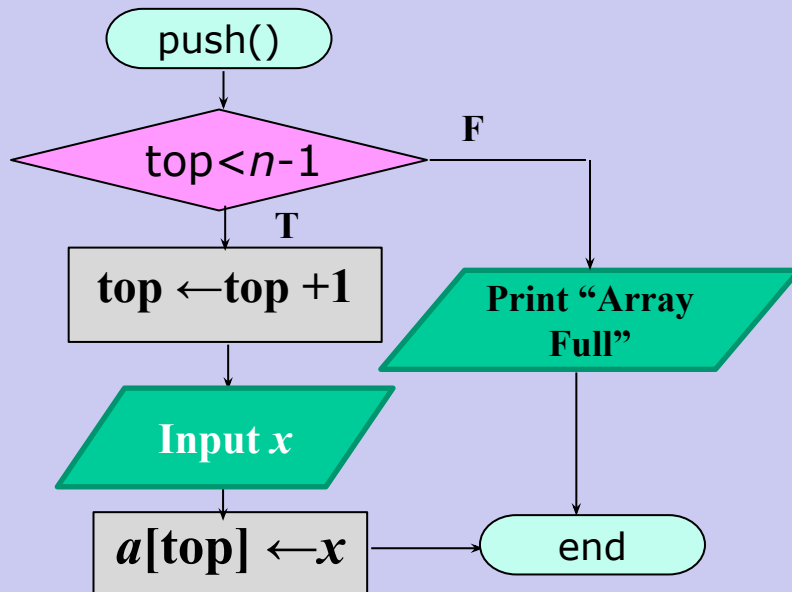
n : size of $a[]$
 x : input variable
 top : index of last input, declare it global.
Array Elements: $a[0] \dots a[n-1]$

$a[]$	
$n-1$	
...	
...	
2	
1	
0	10 ← top

After 1st element (10) push, $top = 0$

Stack: **push()** function

Topic 1: Write an Algorithm to push a new element in a stack



n : size of $a[]$
 x : input variable
 top : index of last input, declare it global.
Array Elements: $a[0] \dots a[n-1]$

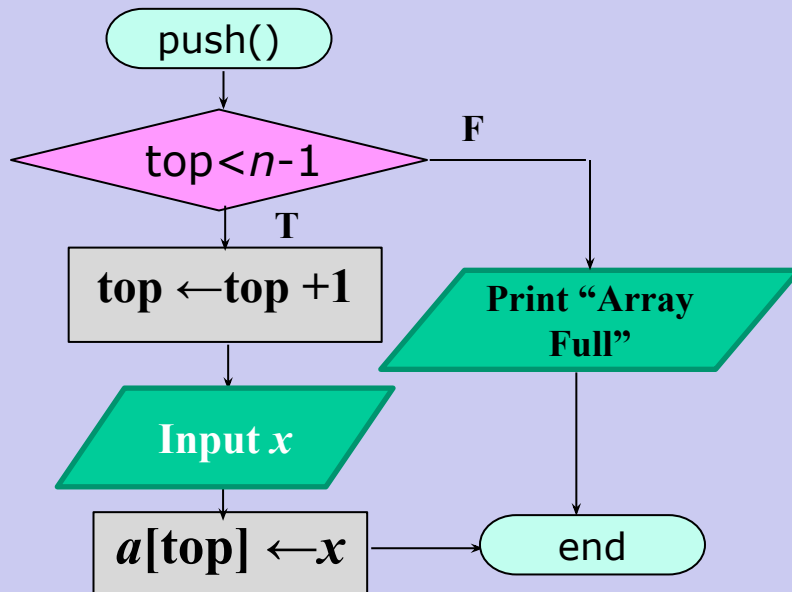
$a[]$	
$n-1$	
...	
...	
2	
1	30
0	10

← top

After 2nd element (30) push, $top = 1$

Stack: **push()** function

Topic 1: Write an Algorithm to push a new element in a stack



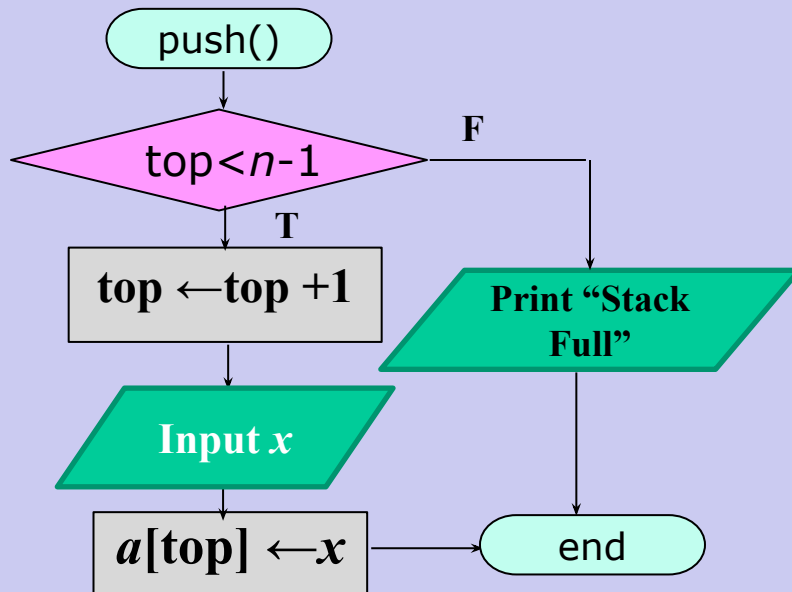
n : size of $a[]$
 x : input variable
 top : index of last input, declare it global.
Array Elements: $a[0] \dots a[n-1]$

$a[]$		
$n-1$		← top
...		
...		
2		
1	30	
0	10	

After last element (30) push, $top = n-1$

Stack: **push()** function

Topic 1: Write an Algorithm to push a new element in a stack



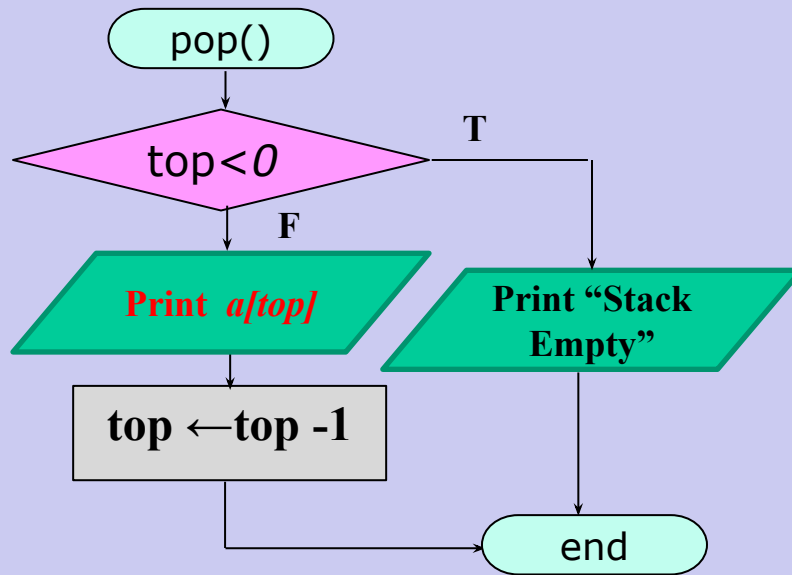
n : size of $a[]$
 x : input variable
 top : index of last input, declare it global.
Array Elements: $a[0] \dots a[n-1]$

$a[]$		
$n-1$		←top
...		
...		
2		
1	30	
0	10	

Try to push more get “**Stack Full**” message

Stack: **pop()** function

Topic 1: Write an Algorithm to pop element from a stack



Suppose `top = 1`

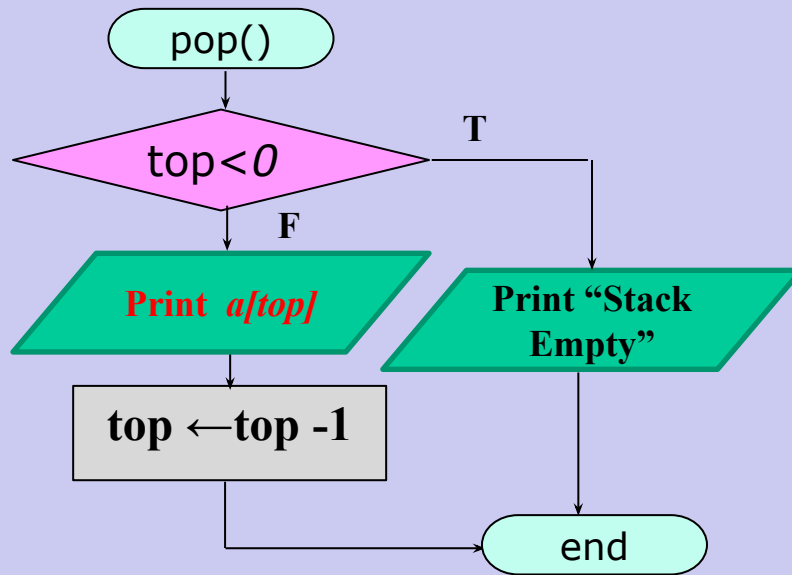
n : size of $a[]$
 top : index of last input, declare it global.
Array Elements: $a[0] \dots a[n-1]$

$a[]$	
$n-1$	
...	
...	
2	
1	40
0	10

$\leftarrow top$

Stack: pop() function

Topic 1: Write an Algorithm to pop element from a stack



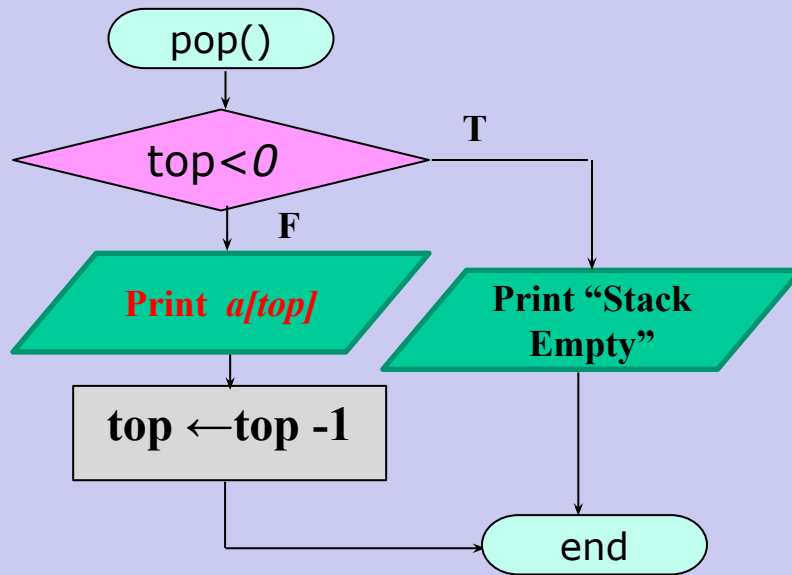
n : size of $a[]$
 top : index of last input, declare it global.
Array Elements: $a[0] \dots a[n-1]$

$a[]$	
$n-1$	
...	
...	
2	
1	40
0	10
$\leftarrow top$	

After deletion 2nd element, $top=0$

Stack: **pop()** function

Topic 1: Write an Algorithm to pop element from a stack



n : size of $a[]$
 top : index of last input, declare it global.
Array Elements: $a[0] \dots a[n-1]$

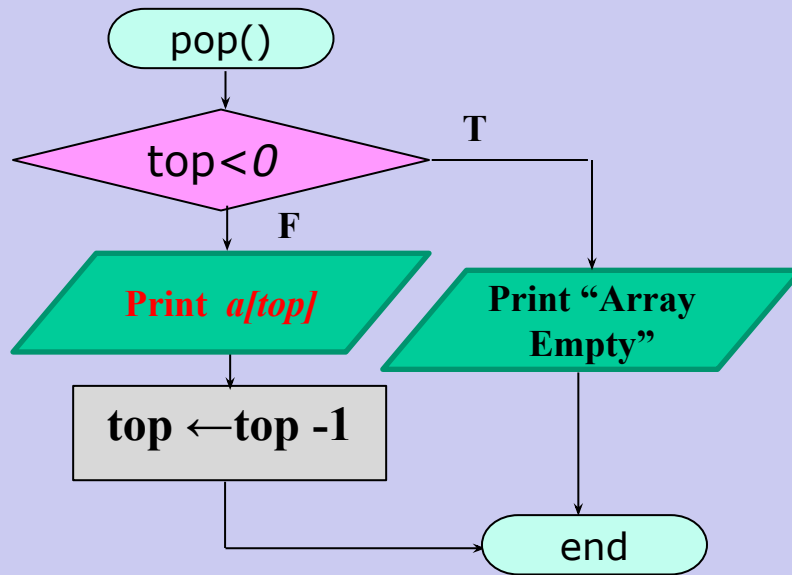
$a[]$	
$n-1$	
...	
...	
2	
1	40
0	10

After deletion 1st element, $top = -1$

$\leftarrow top$

Stack: **pop()** function

Topic 1: Write an Algorithm to pop element from a stack



n : size of $a[]$
 top : index of last input, declare it global.
Array Elements: $a[0] \dots a[n-1]$

$a[]$	
$n-1$	
...	
...	
2	
1	40
0	10

Try to delete more gives message **"Stack Empty"**

← top

Polish Notation

What is Polish Notation?

Polish notation is a way of expressing arithmetic expression that avoids the use of brackets to define priorities for evaluation of operators.

Types of Polish Notation

1. **Infix** □ Operator lies between two operands i.e. $2+3$
2. **Prefix** □ Operator lies before two operands i.e. $+23$
3. **Postfix** □ Operator lies after two operands i.e. $23+$

Polish Notation

Infix: $1+2 \times 3+1/2 = ?$

~~$((1+(2 \times 3))+1)/2=7/2=3.5$~~

Postfix (Polish notation):

Let A, B, be operands, ♦ an operator.

Instead of $A \diamond B$, write $AB \diamond$

Polish Notation

Example:

Infix:

$$((1+2) \times (3+1)) / 2$$

Postfix:

$$((1+2) \times (3+1)) 2 /$$

$$(1+2)(3+1) \times 2 /$$

$$(1+2) 3 1 + \times 2 /$$

$$1 2 + 3 1 + \times 2 /$$

Infix:

$$((1+2) \times (3+1)) / 2$$

Prefix:

$$/((1+2) \times (3+1)) 2$$

$$/ \times (1+2)(3+1) 2$$

$$/ \times + 1 2 (3+1) 2$$

$$/ \times + 1 2 + 3 1 2$$

Polish Notation

Advantage of Polish Notation:
No ambiguity!

$A-B-C = ?$

$(A-B)-C$

$A-(B-C)$

Order of Operations / Operator Precedence -

- 1) Parentheses - {}, [], ()
- 2) Exponents(Right to Left) - A^B , 2^{3^4}
- 3) Multiplication & Division(Left to Right) - $A*B/C$
- 4) Addition & Subtraction(Left to Right) - $A + B - C$

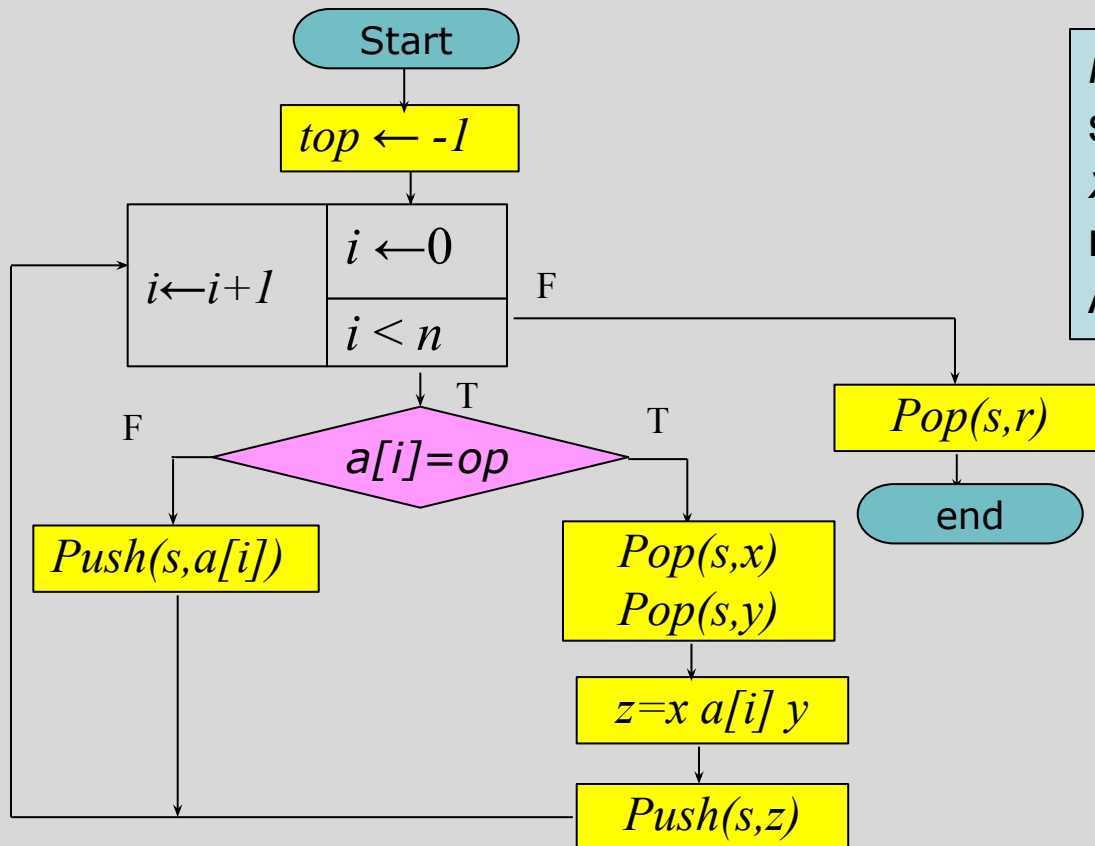
Associativity -

Associativity describes the rule where operators with the same precedence appear in an expression. For example, in expression $a + b - c$, both $+$ and $-$ have the same precedence, then which part of the expression will be evaluated first, is determined by associativity of those operators.

$AB-C-$

$ABC--$

Polish Notation



n : size of $a[]$
 s : stack
 x, y, z : temp variable
 r : store result
 Array Elements: $a[0].....a[n-1]$

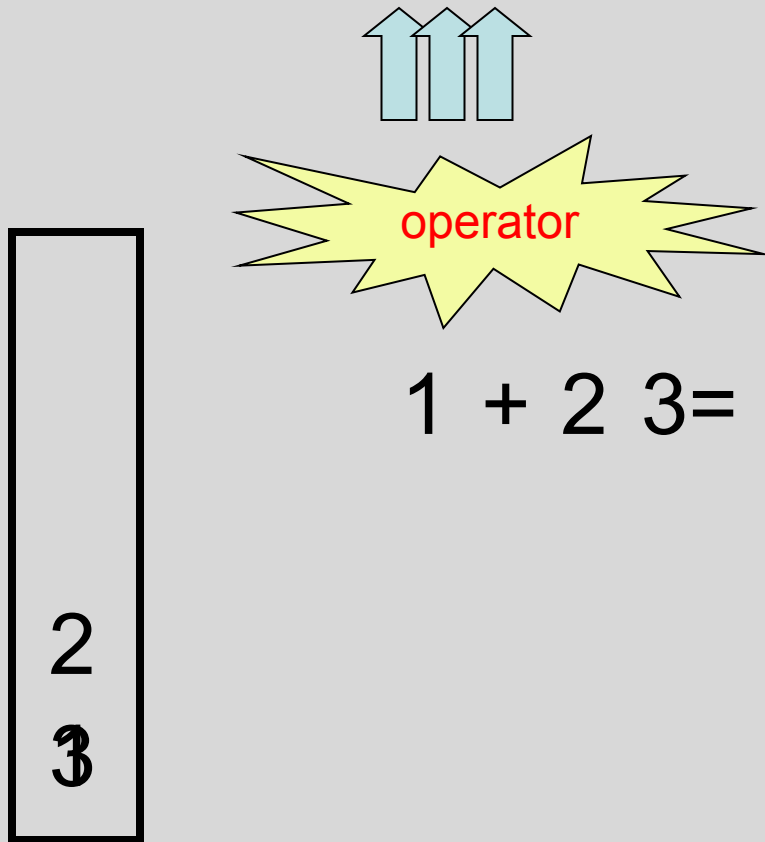
a[]	8	/
	7	2
	6	x
	5	+
	4	1
	3	3
	2	+
	1	2
	0	1

Example: $12+31+x2/ \quad ((1+2)x(3+1))/2=6$

Polish Notation

Example: $12+31+x2/$

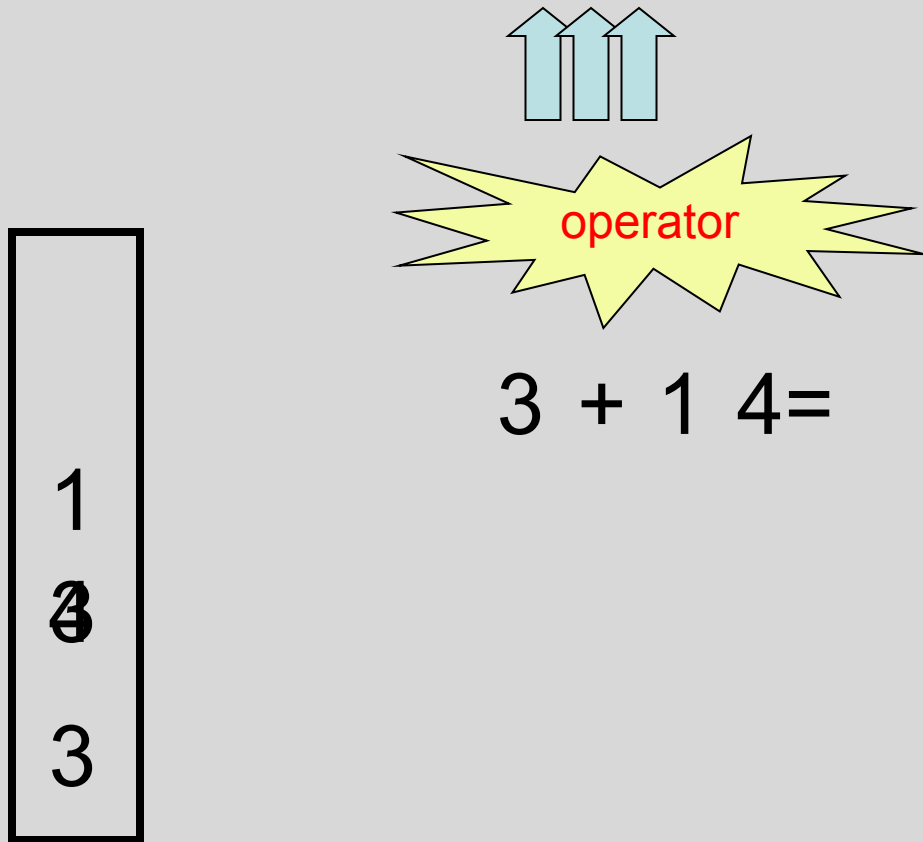
$((1+2)x(3+1))/2=6$



Polish Notation

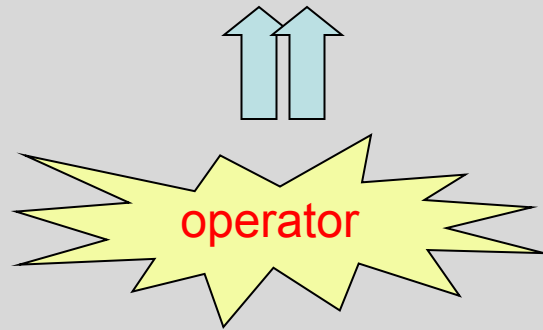
Example: 12+31+x2/

$((1+2) \times (3+1)) / 2 = 6$



Polish Notation

Example: $12+31+x2/$ $((1+2)x(3+1))/2=6$



3 x 4 12=

2
13

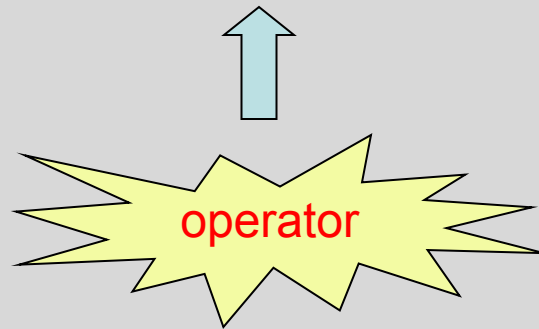
Polish Notation

Example: 12+31+x2/

$((1+2) \times (3+1)) / 2 = 6$

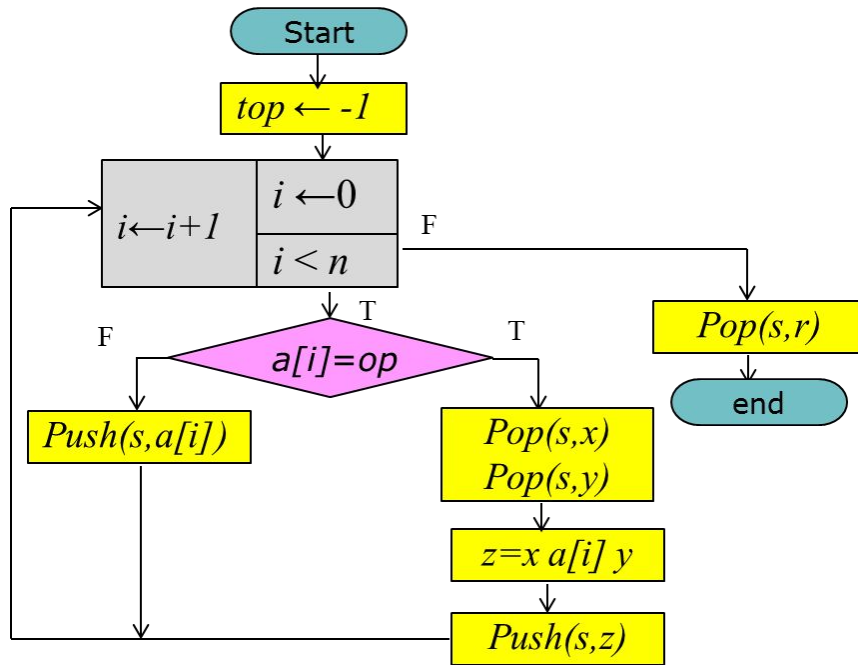
6

2
12



12 / 2 6=

Polish Notation



The screenshot shows a terminal window titled "D:\CodeBlocks\polish\bin\Debug\polish.exe". The user enters the expression "12+31+*2/". The program outputs "The result of expression 12+31+*2/ = 6". At the bottom, it shows "Process returned 0 (0x0) execution time : 49.245 s" and "Press any key to continue."

```
#include <iostream>
using namespace std;
```

```
class Polish{
    int stack[20];
    int top = -1;
public:
    void push(int x)
    { stack[++top] = x; }
    int pop()
    { return stack[top--]; }
```

```
void PostFix(char *e){
    int n1,n2,n3,num;
```

```
    while(*e != '\0')
    {
        if(isdigit(*e))
        {
            num = *e - 48;
            push(num);
        }
        else
        {
            n1 = pop();
            n2 = pop();
            switch(*e)
            {
                case '+': n3 = n1 + n2; break;
                case '-': n3 = n2 - n1; break;
                case '*': n3 = n1 * n2; break;
                case '/': n3 = n2 / n1; break;
            }
            push(n3);
        }
        e++;
    }
    cout<<"="<<pop();
}
};
```

```
int main()
```

```
{
    char exp[20];
    char *e;
    cout<<"Enter the expression :: ";
    cin>>exp;
    Polish p;
    cout<<"\nThe result of expression "<<exp;
    p.PostFix(exp);
    return 0;
}
```

Assignments

Prob 1: An array `c[]` stores characters (alphabets and digits) then write an algorithm that creates two stacks to store alphabets and digits respectively from `c[]`.

Prob 2: Write an algorithm that converts an infix expression to its prefix equivalent